

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №3

по дисциплине

«Микропроцессорные и встраиваемые системы»

Выполнили: **Салженикин Д.А.**
Галиев Э.Р.
Группа: **ИУ1-52Б**
Проверил: **Сычёв В.В.**

Работа выполнена: **02.11.2023**
Отчет сдан: **14.12.2023**
Оценка:

Москва 2023

Содержание

Цель работы: 3
Н-мост 3
Устройство и принцип работы драйвера 4
Код 5

Цель работы:

Настройка работы моторчика при помощи stm32, h-моста и макетной платы.

H-мост

H-мост — это электронная схема, которая даёт возможность приложить напряжение к нагрузке в разных направлениях. Эта схема очень часто используется в робототехнике и игрушечных машинах, чтобы изменять направление вращения мотора. H-мосты представлены в виде интегральных схем, а также могут быть построены из отдельных радиодеталей.

Устройство и принцип работы драйвера

Драйвер — компьютерное программное обеспечение, с помощью которого другое программное обеспечение (операционная система) получает доступ к аппаратному обеспечению некоторого устройства. Обычно с операционными системами поставляются драйверы для ключевых компонентов аппаратного обеспечения, без которых система не сможет работать. Однако для некоторых устройств (таких, как видеокарта или принтер) могут потребоваться специальные драйверы, обычно предоставляемые производителем устройства. Пример драйвера h-моста, используемого в лабораторной работе представлен на рисунке 1.

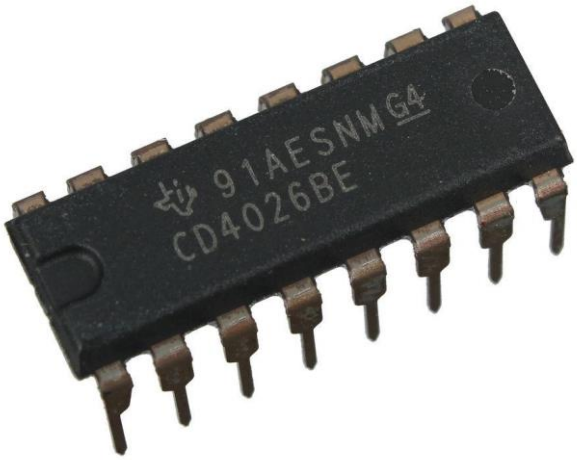


Рисунок 1 — Драйвер h-моста

Принцип работы драйвера Н-моста заключается в том, что он даёт возможность приложить напряжение к нагрузке в разных направлениях. Драйвер Н-моста также отвечает за обновление таблицы адресов и обнаружение новых устройств в сети.

В целом, Н-мост и его драйвер работают в тесном взаимодействии, чтобы обеспечить эффективную передачу данных между различными сетями.

Код

```
	#include "main.h"
	/* Private variables ---------------------------------------------------
------*/
	 ADC_HandleTypeDef hadc1;
	TIM_HandleTypeDef htim1;
	UART_HandleTypeDef huart2;

	/* USER CODE BEGIN PV */
	uint32_t countOfTIMoverflow;
	uint32_t currentStateTIM;
	uint32_t encoderPeriod;
	uint32_t oldStateTIM;
	int16_t angle;
	/* USER CODE END PV */

	/* Private function prototypes -----------------------------------------
------*/
	void SystemClock_Config(void);
	static void MX_GPIO_Init(void); //пинов
	static void MX_ADC1_Init(void); //ацп
	static void MX_TIM1_Init(void); //таймера
	static void MX_USART2_UART_Init(void);
	/* USER CODE BEGIN PFP */

	int main(void)
	{
	  /* Reset of all peripherals, Initializes the Flash interface and the
Systick. */
	  HAL_Init(); //после объявления библиотеки можем использовать только её
```

```
        /* Configure the system clock */
        SystemClock_Config();

        /* Initialize all configured peripherals */
        MX_GPIO_Init();  //иниц таймера
        MX_ADC1_Init();
        MX_TIM1_Init();
        MX_USART2_UART_Init();
        /* USER CODE BEGIN 2 */
        countOfTIMoverflow = 0;
        currentStateTIM =0;  //текущее время
        oldStateTIM = 0; // предыдущее время
        uint32_t adcValue;
        uint32_t str[31];
        HAL_GPIO_WritePin(GPIOE,GPIO_PIN_10,GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOE,GPIO_PIN_12,GPIO_PIN_RESET);
        uint32_t temp;
        HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2); //запуск таймера (ШИМ)

        /* USER CODE BEGIN WHILE */
         while (1)
  {
//    здесь был UART
        for (uint16_t i = 0; i < 3900; i += 100){
              TIM1->CCR2 = i;
              HAL_Delay(200);
        }
        for (uint16_t i = 4000; i > 100; i -= 100){
              TIM1->CCR2 = i;
              HAL_Delay(200);
        }
        TIM1->CCR2 = 0;
        HAL_GPIO_TogglePin(GPIOE,GPIO_PIN_10); //инверсия
        HAL_GPIO_TogglePin(GPIOE,GPIO_PIN_12); //инверсия
  }
}
      void SystemClock_Config(void)
      {
        RCC_OscInitTypeDef RCC_OscInitStruct = {0};
        RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
        /** Configure the main internal regulator output voltage
        */
        if  (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1)  !=
HAL_OK)
        {
```

```
    Error_Handler();
  }
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
  RCC_OscInitStruct.MSIState = RCC_MSI_ON;
  RCC_OscInitStruct.MSICalibrationValue = 0;
  RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_MSI;
  RCC_OscInitStruct.PLL.PLLM = 1;
  RCC_OscInitStruct.PLL.PLLN = 38;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
  RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
  RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK){
    Error_Handler();
  }
  /** Initializes the CPU, AHB and APB buses clocks */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if   (HAL_RCC_ClockConfig(&RCC_ClkInitStruct,   FLASH_LATENCY_4)   !=
HAL_OK){
    Error_Handler();
  }
}
static void MX_ADC1_Init(void)
{
  ADC_MultiModeTypeDef multimode = {0};
  ADC_ChannelConfTypeDef sConfig = {0};
  /** Common config */
  hadc1.Instance = ADC1;
  hadc1.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV1;
  hadc1.Init.Resolution = ADC_RESOLUTION_12B;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
  hadc1.Init.LowPowerAutoWait = DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
```

```
  hadc1.Init.NbrOfConversion = 1;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
  hadc1.Init.DMAContinuousRequests = DISABLE;
  hadc1.Init.Overrun = ADC_OVR_DATA_PRESERVED;
  hadc1.Init.OversamplingMode = DISABLE;
  if (HAL_ADC_Init(&hadc1) != HAL_OK){
    Error_Handler();
  }
  /** Configure the ADC multi-mode */
  multimode.Mode = ADC_MODE_INDEPENDENT;
  if (HAL_ADCEx_MultiModeConfigChannel(&hadc1, &multimode) != HAL_OK) {
    Error_Handler();
  }
  /** Configure Regular Channel */
  sConfig.Channel = ADC_CHANNEL_6;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_2CYCLES_5;
  sConfig.SingleDiff = ADC_SINGLE_ENDED;
  sConfig.OffsetNumber = ADC_OFFSET_NONE;
  sConfig.Offset = 0;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK){
    Error_Handler();
  }
}
static void MX_TIM1_Init(void) {
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};

  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 0;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 4096;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim1) != HAL_OK){
    Error_Handler();
  }
```

```
        sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
        if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
        {
          Error_Handler();
        }
        if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
        {
          Error_Handler();
        }
        sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
        sMasterConfig.MasterOutputTrigger2 = TIM_TRGO2_RESET;
        sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
       if  (HAL_TIMEx_MasterConfigSynchronization(&htim1,  &sMasterConfig)  !=
HAL_OK) {
          Error_Handler();
        }
        sConfigOC.OCMode = TIM_OCMODE_PWM1;
        sConfigOC.Pulse = 0;
        sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
        sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
        sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
        sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
        sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
       if  (HAL_TIM_PWM_ConfigChannel(&htim1,  &sConfigOC,  TIM_CHANNEL_2)  !=
HAL_OK) {
          Error_Handler();
        }
        sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
        sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
        sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
        sBreakDeadTimeConfig.DeadTime = 0;
        sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
        sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
        sBreakDeadTimeConfig.BreakFilter = 0;
        sBreakDeadTimeConfig.Break2State = TIM_BREAK2_DISABLE;
        sBreakDeadTimeConfig.Break2Polarity = TIM_BREAK2POLARITY_HIGH;
        sBreakDeadTimeConfig.Break2Filter = 0;
        sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
       if  (HAL_TIMEx_ConfigBreakDeadTime(&htim1,  &sBreakDeadTimeConfig)  !=
HAL_OK) {
          Error_Handler();
        }
```

```
  HAL_TIM_MspPostInit(&htim1);
}
static void MX_USART2_UART_Init(void) {
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 19200;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
  huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
  if (HAL_UART_Init(&huart2) != HAL_OK){
    Error_Handler();
  }
}
static void MX_GPIO_Init(void){
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOE_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOE, GPIO_PIN_10|GPIO_PIN_12, GPIO_PIN_RESET);

  /*Configure GPIO pin : PA0 */
  GPIO_InitStruct.Pin = GPIO_PIN_0;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLDOWN;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : PE10 PE12 */
  GPIO_InitStruct.Pin = GPIO_PIN_10|GPIO_PIN_12;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

  /*Configure GPIO pin : PE13 */
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
```

```
  GPIO_InitStruct.Pull = GPIO_PULLDOWN;
  HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

  /*Configure GPIO pin : PE14 */
  GPIO_InitStruct.Pin = GPIO_PIN_14;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
}

/* USER CODE BEGIN 4 */
 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
currentStateTIM =TIM1->CNT;
encoderPeriod = currentStateTIM - oldStateTIM;
oldStateTIM = currentStateTIM;
if(HAL_GPIO_ReadPin(GPIOE,GPIO_PIN_14)==GPIO_PIN_SET) {
      angle++; }
else {
      angle--; }
}
/* USER CODE END 4 */

void Error_Handler(void){
  __disable_irq();
  while (1)  {  }
}

#ifdef  USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line){ }
#endif /* USE_FULL_ASSERT */
```